

オープン保育ICTをTrueNASへ導入する手順書

初心者向け 初回導入から保護者の利用開始まで

作成日：2026年9月6日 手順書 第1版

対象コード：mainの 1cc7dc83ea37e572282e67ea461d0424abd328a0

この手順書は、TrueNASに園児や職員のデモデータが入っていないオープン保育ICTを用意し、施設のURLから使い始めるためのものです。今回の実機導入で行った作業と、そこでつまづいた点をまとめました。

最初の到達点は、管理者がログインし、架空の園児1名について保護者への招待、初回入力、園の承認、保護者のログインまでを確認することです。その後、端末通知とバックアップを確認します。

すでに稼働している環境では、初回導入の章をやり直さないでください。更新は第18章へ進みます。 保存先を作り直したり、デモデータを投入したりする操作は必要ありません。

本書はベータ版の実機検証用です。最初は架空の園児名と、作業担当者が受信できるメールアドレスを使います。実際の業務で使い始める時点では、バックアップからの復元と、施設の運用体制まで確認してください。

この本の進め方

章	作業	操作する場所
1～3	仕組みを知り、接続先と設定値を準備する	手元のPCとTrueNAS
4～8	保存先、アプリ、メール、秘密情報を用意する	SSH端末
9～11	Cloudflareを設定し、アプリと管理者を有効化する	ブラウザーとSSH端末
12～15	クラス、保護者の初回登録、家族との紐付けを確認する	アプリの画面
16～17	端末通知を設定して受信を試す	SSH端末と保護者端末
18～21	バックアップ、更新、困ったときの確認、導入記録	担当者全員

各章の「完了の目安」を確認してから、次へ進みます。コマンドはMarkdown版からコピーしてください。PDFでは長い行が折り返される場合があります。

1 全体の仕組みを知る

保護者や職員は、TrueNASの管理画面ではなく、施設用のURLを開きます。

職員や保護者のブラウザー
↓ 施設用のHTTPS URL
Cloudflare Access 利用を許可した人か確認
↓ Cloudflare Tunnel
TrueNAS上のアプリ アプリのIDとパスワードで確認
├─ 園児や職員のデータと添付を保存
├─ Gmail経由で招待メールを送信
└─ 通知サービス経由で端末へプッシュ通知

最初に覚える言葉

言葉	本書での意味
SSH	手元のPCからTrueNASへ命令を送る接続方法
Docker	アプリをコンテナという独立した単位で動かす仕組み
イメージ	コンテナを作るためのアプリ一式
Compose	複数のコンテナと設定をまとめて起動する仕組み
Dockge	Composeの設定や稼働状況を見る管理画面
スタック	今回のアプリと関連コンテナをまとめた構成
プールとデータセット	TrueNASの大きな保存領域と、その中の管理単位
runtime	日々変化するDBと添付ファイルの保存場所
SMTP	アプリがメールを送るための接続方法
VAPID	プッシュ通知の送信元を識別する鍵の仕組み
コミット番号	導入するコードの版を示す、40文字の識別番号

Cloudflareの認証と、アプリのログインは別です。Cloudflareを通過しても、アプリの職員・保護者アカウントは自動では作られません。

施設のアプリURLと、TrueNAS・Dockgeの管理URLも別です。TrueNASとDockgeは管理用のLANやVPNから操作します。

2 導入前の準備と記入表

この手順は、Dockerを利用できるTrueNASと、設定済みのDockgeがある状態から始めます。今回の確認環境はTrueNAS 25.10.1、Dockge 1.5.0です。TrueNAS COREや古いKubernetes方式のAppsへ、このまま適用することはできません。

Dockgeが未導入の場合は、TrueNASのAppsから導入し、スタックの保存先を確認してから進みます。標準AppsのYAML入力欄と、DockgeのCompose設定は同じ操作ではありません。[TrueNAS公式のCustom Apps案内](#)

用意するもの	自分の環境の記入欄
TrueNASの管理画面URL	_____
SSHの接続先とユーザー	_____
Dockgeの管理画面URL	_____
Dockgeのホスト側スタック保存先	_____
使用するプール名	_____
新しいデータセット名	_____
施設用のホスト名	_____
送信に使うGmailアドレス	_____
管理者の連絡先メール	_____
外部にも保管するバックアップ先	_____
作業担当者と、困ったときの連絡先	_____

Cloudflareのアカウントと、そこでDNSを管理しているドメインを用意します。例えば `pilot.example.com` を施設用ホスト名にします。`example.com` は説明用なので、自分のドメインへ置き換えます。

パスワード、Gmailのアプリパスワード、Tunnelのトークン、通知の秘密鍵は、この記入表には書きません。
パスワード管理ツールなど、施設で決めた保管場所を使います。

完了の目安：管理画面を開けること、SSH接続に必要な情報があること、メールを実際に受信できることを確認しました。

3 SSHで接続して現在の状態を確認する

操作する場所：最初は手元のWindows PC、その後はTrueNASのSSH端末。

Windowsで「ターミナル」またはPowerShellを開きます。次のIPアドレスは例なので、自分の接続先へ置き換えます。VPN経由の場合は、先にVPNへ接続します。

```
ssh truenas_admin@192.168.1.10
```

初回到接続先の確認が出たら、TrueNASの管理担当者とホスト鍵の指紋を照合してから接続を承認します。パスワード入力中に文字や「*」が表示されないのは正常です。入力後にEnterを押します。

truenas_admin@truenas のような表示に変わったら接続できています。本書のコマンドを同じ書き方で使うため、続けてBashへ切り替えます。

```
bash
sudo docker ps --format '{{.Names}}\t{{.Image}}\t{{.Ports}}'
sudo docker compose version
sudo zpool list
```

sudo は管理者権限で実行する指定です。求められたパスワードは、このSSHユーザーのもので、GmailやCloudflareのパスワードではありません。

結果の見方

- `docker ps` : 現在動いているコンテナが並びます。既存アプリの名前とポートを控えます。
- `docker compose version` : Composeのバージョンが表示されます。
- `zpool list` : 使うプールのHEALTHが **ONLINE** で、必要な空き容量があることを確認します。

コマンドには、画面に出ている truenas_admin@...\$ まで含めて貼り付けません。本文中の `app_mounts` や `@` に、装飾用の余計なバックスラッシュを付けないでください。

完了の目安：SSHで操作でき、既存コンテナと保存領域を確認できました。

4 新しい保存先を用意する

操作する場所：TrueNASの管理画面とSSH端末。初回導入だけの作業です。

TrueNASの「Datasets」で、使用するプールの下に `open-hoikuict-pilot` を新規作成します。その下に `runtime`、`secrets`、`backups` を作成します。既存の同名データセットがあれば、新規作成を進めず用途を確認します。

```
main/open-hoikuict-pilot
runtime  DBと添付をまとめて保存
secrets  秘密情報と禁止パスワード一覧
backups  初期検証用のバックアップ
```

この例の `main` はプール名です。Gitのmainブランチとは関係ありません。`backups` は同じプールにあるため、後で別の機器や保存先にも複製します。

SSHで次の変数を設定します。これらは入力を短くするための略称です。**同じSSH接続内で使用し、接続し直したら再設定します。** `STACKS` は自分のDockgeの保存先へ変更してください。

```
BASE=/mnt/main/open-hoikuict-pilot
STACKS=/mnt/.ix-apps/app_mounts/dockge/stacks
STACK="$STACKS/open-hoikuict-pilot"
SHA=1cc7dc83ea37e572282e67ea461d0424abd328a0
IMAGE="open-hoikuict:$SHA"
```

Dockgeの設定では、スタックのパスをホスト側とコンテナ側で一致させます。上の `.ix-apps` を含むパスは今回の実機の例で、全環境共通ではありません。[Dockge公式の保存先の説明](#)

新規・空のデータセットであることを確認してから実行します。

```
sudo install -d -m 755 "$BASE/source" "$STACK"
sudo install -d -m 750 "$BASE/runtime/data" "$BASE/runtime/storage"
sudo chmod 700 "$BASE/secrets"
```

DBと添付はローカルのZFS上に置きます。これらのフォルダーをSMB/NFS上のDB保存先として使わず、開発PCのDBや `.env` をコピーしません。権限エラーが出たら、対象データセットのACLを確認します。プール全体への権限変更は行いません。

5 導入する版を固定してイメージを作る

操作する場所：TrueNASのSSH端末。

第4章の変数を設定した同じ端末で進めます。以下の open-hoikuict-release フォルダーがすでにあれば、再利用せず中身を確認してください。

```
git clone https://github.com/hoikuict/open-hoikuict.git ~/open-hoikuict-release
git -C ~/open-hoikuict-release checkout --detach "$SHA"
git -C ~/open-hoikuict-release rev-parse HEAD
git -C ~/open-hoikuict-release status --short
```

表示された40文字の番号が第4章のSHAと一致し、最後のコマンドが何も表示しないことを確認します。Gitが利用できない場合は、TrueNASへ勝手に追加パッケージを入れず、管理担当者に同じ版のソースアーカイブを用意してもらいます。

空の source へ、配布対象のファイルだけを展開します。

```
git -C ~/open-hoikuict-release archive --format=tar "$SHA" > ~/open-hoikuict-source.tar
sudo tar -xf ~/open-hoikuict-source.tar -C "$BASE/source"
sudo cp -n "$BASE/source/deploy/truenas/compose.yaml" "$STACK/compose.yaml"
sudo cp -n "$BASE/source/deploy/truenas/.env.example" "$STACK/.env"
sudo chmod 600 "$STACK/.env"
sudo docker build --label "org.opencontainers.image.revision=$SHA" -t "$IMAGE" "$BASE/source"
```

ビルドは、アプリを動かすための一式を作る作業です。処理中は画面が流れます。エラーで停止した場合は、その先の起動へ進みません。

続けて、作成結果とアプリの実行ユーザー番号を調べます。

```
sudo docker image inspect "$IMAGE" --format '{{.Id}}'
APP_UID=$(sudo docker run --rm --network none --entrypoint id "$IMAGE" -u)
APP_GID=$(sudo docker run --rm --network none --entrypoint id "$IMAGE" -g)
sudo chown "$APP_UID:$APP_GID" "$BASE/runtime/data" "$BASE/runtime/storage" "$BASE/backups"
sudo chmod 750 "$BASE/runtime/data" "$BASE/runtime/storage" "$BASE/backups"
sha256sum "$STACK/compose.yaml"
```

sha256:... のイメージIDとComposeのハッシュを控えます。完了の目安：イメージが作成され、DBと添付の保存先へアプリが書き込める権限を用意できました。

6 設定ファイルへ施設の値を入れる

操作する場所：SSH端末。設定ファイルはスタック直下の `.env` です。

```
cd "$STACK"
sudo vi .env
```

`vi`は文字を編集する道具です。`i`で入力を開始し、編集後に`Esc`、`:wq`、`Enter`で保存して終了します。保存せず戻る場合は`Esc`、`:q!`、`Enter`です。操作に不安がある場合は、管理担当者と一緒に編集します。

`<...>` が付いた仮の値を、すべて自分の値へ置き換えます。値は `項目名='値'` の形にします。`.env`の全文を相談用チャットや画像に載せません。

項目	入れる値
COMPOSE_PROJECT_NAME	open-hoikuict-pilot
APP_IMAGE	open-hoikuict: に第4章の40文字のSHAを続けた値
APP_RUNTIME_PATH	第4章のBASEに <code>/runtime</code> を続けたパス
APP_SECRETS_PATH	BASEに <code>/secrets</code> を続けたパス
BACKUP_SET_PATH	BASEに <code>/backups</code> を続けたパス
BACKUP_GIT_SHA	第4章のSHA
BACKUP_APP_IMAGE	第5章で確認した <code>sha256:...</code>
BACKUP_COMPOSE_SHA256	第5章のComposeハッシュ
PILOT_HOSTNAME	自分の施設用ホスト名。 <code>https://</code> や <code>/</code> は付けない
PILOT_NETWORK_CIDR	他と重複しない専用範囲。今回の例は <code>172.30.50.0/24</code>
HOIKU_NURSERY_REF	施設を識別する名前
CLOUDFLARED_IMAGE	第8章で取得する固定イメージの識別値

ネットワーク範囲は、LAN・VPN・既存のDockerネットワークと重ならないようにします。`sudo docker network ls`、各ネットワークの `sudo docker network inspect 名前`、`ip route` で管理担当者と確認します。

第7章でSMTPと2つのランダム値を設定します。Composeの本番用認証、HTTPS、CSRFの設定は雛形に入っています。設定エラーを避けるために認証を無効化する必要はありません。

7 Gmailとアプリの秘密情報を設定する

操作する場所：Googleアカウントの画面とSSH端末。

Gmailを準備する

送信用のGoogleアカウントへログインし、2段階認証を有効にします。そのアカウントの「アプリ パスワード」を開き、オープン保育ICT用のものを作成します。作成できない場合は、アカウントの管理設定や保護設定を確認します。[Googleのアプリパスワード案内](#)

.env に次を設定します。送信元・ユーザー名は同じGmailアドレスにそろえます。

項目	設定値
HOIKUICT_SMTP_HOST	smtp.gmail.com
HOIKUICT_SMTP_PORT	587
HOIKUICT_SMTP_USERNAME	送信に使うGmailアドレス
HOIKUICT_SMTP_PASSWORD	作成したアプリパスワード。表示上の区切り空白を除く
HOIKUICT_PARENT_MAIL_FROM	送信に使うGmailアドレス

STARTTLSは接続後に通信を暗号化する方式で、Compose側で有効になっています。Googleの通常のログインパスワードは使いません。[GoogleのSMTP接続設定](#)

アプリ用のランダム値を作る

次を1回実行すると、異なる2つの値が端末に表示されます。画面共有や端末のログ保存をしていない状態で実行し、施設の秘密情報保管先へ保存します。

```
python3 -c 'import secrets; print(secrets.token_urlsafe(48)); print(secrets.token_urlsafe(48))'
```

1行目を .env の HOIKUICT_SECRET_KEY、2行目を HOIKUICT_LOGIN_THROTTLE_HMAC_KEY に入れます。アプリを再起動するたびに作り直す値ではありません。

保存後は `sudo chmod 600 .env` を実行します。Mailpitはテスト用の受信箱で、外部の保護者には届きません。今回のように実際にメールを送る構成では、アプリの接続先をGmailにします。

8 禁止パスワード一覧とTunnelの接続準備

操作する場所：SSH端末とCloudflareの画面。

禁止パスワード一覧を置く

推測されやすいパスワードを拒否するため、空でない一覧ファイルが必要です。導入担当者が確認したUTF-8のテキストを `~/password-blocklist.txt` として用意します。1行に1つの禁止パスワードを記載します。

今回使用した配布元は [SecListsの一般的なパスワード一覧](#) です。GitHubのRaw表示からファイルを保存し、手元のPCから `scp` ファイル名 SSH接続先:~/password-blocklist.txt で転送できます。公開された一覧なので秘密情報ではありません。

```
sudo install -o "$APP_UID" -g "$APP_GID" -m 400 ~/password-blocklist.txt "$BASE/secrets/password-blocklist.txt"
```

Cloudflare用のイメージを固定する

```
sudo docker pull cloudflare/cloudflared:latest
sudo docker image inspect cloudflare/cloudflared:latest --format '{{index .RepoDigests 0}}'
```

表示された `cloudflare/cloudflared@sha256:...` 全体を `.env` の `CLOUDFLARED_IMAGE` に入れます。最初の取得後はこの固定値を使います。

Tunnelを作成してトークンを保存する

CloudflareのZero Trustで「Networks」「Connectors」「Cloudflare Tunnels」などの名前の画面を開き、新しいcloudflaredのTunnelを作ります。名前は `open-hoikuict-pilot` など、用途が分かるものにします。Dockerの接続案内に表示されるトークンを使います。

次はトークンだけを入力するコマンドです。Dockerコマンド全体は貼り付けません。入力内容は画面に表示されず、新規ファイルとして保存されます。

```
sudo python3 -c 'import getpass,os,sys; from pathlib import Path; p=Path(sys.argv[1]); v=getpass.getpass("Tunnel token: ").strip(); assert len(v)>50 and not any(c.isspace() for c in v), "Token only"; f=os.open(p,os.O_WRONLY|os.O_CREAT|os.O_EXCL,0o400); os.fchown(f,65532,65532); os.write(f,v.encode("ascii")); os.close(f); print("Saved")' "$BASE/secrets/tunnel-token"
```

File exists は上書きを止めた表示です。トークンを削除してやり直す前に、既存設定を確認します。Tunnelは次章のAccessを設定した後で起動します。

9 Cloudflareで施設URLと利用者を設定する

操作する場所：Cloudflareの管理画面。

先にAccessで利用者を許可する

Accessの「Applications」から、公開ホスト名を保護するSelf-hostedアプリケーションを追加します。施設用のホスト名を指定し、サイト全体を対象にするためパスは空欄にします。

Allowポリシーで、最初に管理者と検証用保護者のメールアドレスを個別に許可します。メールのワンタイムPINを使う場合は、それをログイン方法として有効にします。後から保護者を招待するときも、Access側の許可が必要です。アプリの招待だけではAccessの許可リストへ追加されません。[Cloudflareの公開アプリ保護手順](#)

作成したAccessアプリケーションの詳細から、Application AudienceのAUDタグをコピーします。Team nameは チーム名.cloudflareaccess.com の「チーム名」の部分です。

Tunnelのルートを追加する

第8章で作成したTunnelの公開アプリケーション／Routesを開き、次を設定します。

入力欄	入れる値
完全なホスト名	施設用のホスト名。 <code>.env</code> と同じ値
パス	空欄
サービスURL	<code>http://app:8000</code>
Protect with Access	オン
Team name	自分のZero Trustチーム名
Application Audienceタグ	今作ったAccessアプリのAUDタグ

AUD欄は、文字を貼り付けただけでは登録扱いにならないことがあります。Enterまたは画面の追加操作でタグとして確定し、赤いエラーが消えたことを確認して保存します。今回の実機でもここでつまづきました。

[Protect with Accessの意味](#)

app はCompose内のサービス名です。外部URLはHTTPS、Tunnelから同じ内部ネットワークのアプリへはHTTPで接続します。`localhost:8000` に置き換えしないでください。

完了の目安：ホスト名、Accessアプリ、個別許可、Team name、AUDの対応がそろいました。

10 起動前に検査してアプリを起動する

操作する場所：SSH端末。初回管理者を作成する前に行います。

```
cd "$STACK"
sudo docker compose --profile '*' config --quiet
sudo docker compose run --rm --no-deps --entrypoint python app -c 'from security_config import validate_runtime_security; validate_runtime_security(); print("Production settings OK")'
sudo docker compose run --rm --no-deps --entrypoint python app -c 'from pathlib import Path; import tempfile; roots=[Path("/data"),Path("/app/storage")]; assert all(p.is_dir() and not any(p.iterdir()) for p in roots), "Runtime must be empty"; [tempfile.TemporaryFile(dir=p).close() for p in roots]; assert Path("/run/secrets/password-blist.txt").read_text(encoding="utf-8").strip(); print("Fresh runtime OK")'
```

最初のコマンドは、成功すると通常何も表示しません。次の2つでOKが出ることを確認します。空領域の検査は初回だけです。2回目以降はデータがあるため、失敗するのが正常です。

```
sudo docker compose up -d app
sudo docker compose ps
sudo docker compose exec -T app python -c 'import urllib.request; print(urllib.request.urlopen("http://127.0.0.1:8000/healthz", timeout=3).read().decode())'
```

起動直後は `starting` となることがあります。少し待って `ps` を再実行し、`healthy` を確認します。健康状態だけでは空のDBか分からないので、主要な台帳も確認します。

```
sudo docker compose exec -T app python - <<'PY'
import sqlite3
with sqlite3.connect('file:/data/hoikuict.db?mode=ro', uri=True) as db:
    for name in ('users', 'children', 'families', 'classrooms', 'parent_accounts'):
        count = db.execute('SELECT COUNT(*) FROM ' + name).fetchone()[0]
        print(name, count)
        assert count == 0, 'Existing data found; do not initialize again'
print('Empty registers OK')
PY
sudo docker compose up -d cloudflared
```

施設の `https://施設ホスト名/staff/login` を開き、Access認証後に「職員ログイン」が出ることを確認します。新しいシークレットウィンドウでもAccessが要求され、許可していないアドレスが通らないことを確認します。

11 最初の管理者を作成する

操作する場所：SSH端末と施設の職員ログイン画面。

```
cd "$STACK"
sudo docker compose exec app python -m scripts.auth_user bootstrap-admin
```

質問	入力する内容
表示名	画面に表示する名前。例：園長
連絡先メールアドレス	管理者本人が受信できるメール
ログインID	職員ログインに使うID。半角英数字など
作成理由	初期導入のため、などの理由
実行者と承認者	施設の運用に沿った実際の担当者
この内容で作成しますか	内容を確認し、作成する場合だけ yes

作成後に有効化コードが一度だけ表示されます。**職員向けコードの有効期限は30分です。** このコードを他人やチャットへ送らず、本人が次の画面でパスワードを設定します。

```
https://施設ホスト名/staff/activate
```

設定後、/staff/login でログインし、いったんログアウトして再ログインできることを確認します。初期管理者は、ログインのたびに作るものではありません。

日本語入力でエラーが出た場合

今回、端末からの日本語入力で `surrogates not allowed` という文字コードエラーが発生しました。パスワード不一致を意味するエラーではありません。

エラーが出た場合は再作成を繰り返さず、既存の管理者が作られていないか確認します。管理者が未作成なら、日本語をASCIIのJSONとして安全に渡す方法などで再試行します。今回使った補助スクリプトは当時の実機専用であり、別の版へそのまま使うものではありません。担当者へエラー文だけを伝え、トークンやコードは渡しません。

完了の目安：管理者1名でログインでき、園児・クラスはまだ空です。コードが期限切れでもDBを消さず、既存職員の有効化コード再発行を使います。

12 園の基本情報と確認用のアカウントを用意する

操作する場所：職員としてログインしたアプリの画面。

クラスを作る

左のメニューから「クラス管理」を開き、園で使うクラスを登録します。最初の検証では「どんぐり」など、確認用のクラスを1つ作れば十分です。

新入園児の氏名・カナ・住所を、ここでまとめて職員が入力する必要はありません。次章の「保護者に初回入力を依頼」を使うと、園では子どもの名前と保護者のメールだけを入力して開始できます。

職員を追加する場合

「職員管理」で表示名や権限などを登録します。台帳への登録と、パスワードでログインできる状態は別です。認証を有効にする場合は、既存職員を対象に次を実行します。

```
cd "$STACK"
sudo docker compose exec app python -m scripts.auth_user activate-staff
```

対象職員の番号、ログインID、発行理由などを確認し、表示された有効化コードを本人へ施設の決めた方法で渡します。本人が /staff/activate でパスワードを設定します。職員コードの期限は30分です。

検証に使うメールアドレス

保護者役として受信できるメールアドレスを1つ用意し、Cloudflare Accessでも許可します。園児名には「試験はな」など架空の名前を使います。Gmailの送信元と、保護者の宛先は別の役割です。

職員と保護者を同じPCで確認する場合は、別のブラウザタブプロファイルを使うと、どちらで操作しているか区別できます。

完了の目安：クラスが1つあり、管理者として操作でき、招待を受け取る保護者役のメールとAccess許可を用意できました。

13 子どもの名前とメールだけで初回入力を依頼する

操作する場所：最初は園の画面、その後は保護者役の端末。

園が行うこと

1. 「園児一覧」または「保護者アカウント」を開きます。
2. 「保護者に初回入力を依頼」を押します。
3. 子どもの名前と、保護者のメールアドレスを入力します。
4. 宛先を復唱するなどして確認し、「初回入力の招待を送信」を押します。

氏名・カナ・住所・電話・勤務先などを園が先に埋める必要はありません。既存園児の情報を補う場合は、その園児の詳細画面から初回入力を依頼します。同じ子を新規で二重登録しないようにします。

保護者が行うこと

1. 招待メールを開きます。迷惑メールフォルダーも確認します。
2. メールリンクを押し、Cloudflareの認証が出たら許可されたメールで通過します。
3. 子どものカナ・生年月日・入園予定日、保護者の氏名・続柄・住所・電話・勤務先などを入力します。
4. 入力内容を確認して提出し、「初回入力を受け付けました」を確認します。

この時点では園の確認待ちです。パスワード設定の案内は、園が承認した後に別のメールが届きます。

リンクを開けない場合

今回、Cloudflareの認証を挟んだ初回の操作で「リンクを確認できません」と出ました。認証後に元のメールへ戻り、もう一度リンクを押すと進めた事例があります。

新しい版では、初回登録画面の「登録コードまたはメールのリンク」へ、メールに記載された登録コードを貼り付ける方法も使えます。コードのない古いメールでは、本文の元リンクをコピーします。Cloudflareの認証コードとは別のものです。

完了の目安：メールを実際に受信でき、保護者役の初回入力が提出できました。送信履歴の処理完了だけで、受信箱への到着まで確認したことにはなりません。

14 園で提出内容を確認して承認する

操作する場所：管理者としてログインした職員ホーム。

1. ホームを更新し、「すべての承認待ち」を確認します。
2. 「保護者の初回登録」を開きます。
3. 子ども・保護者・招待したメールアドレスの対応、提出内容を確認します。
4. 確認欄にチェックし、理由を記入して承認します。

提出内容は「保護者アカウント」の「確認待ちの初回登録」、または対象者の「認証管理」→「登録申請の履歴」からも開けます。

承認すると反映されるもの

- ・ 園児の情報と、家族プロフィール。
- ・ 保護者アカウントと、その園児を閲覧できる紐付け。
- ・ 保護者へ送る、パスワード設定の案内メール。

保護者は2通目のメールからパスワードを設定します。その後、次の保護者ログインURLでログインします。初回入力の内容をもう一度入力する必要はありません。

`https://施設ホスト名/parent-portal/login`

クラスの人数が増えない場合

承認後、園児の所属クラスと入園予定日を確認します。ホームのクラス人数は、当日の集計条件に合う園児を数えます。

例えば今日が9月6日で、入園予定日が10月1日なら、どんぐりクラスへ登録しても今日の人数は増えません。今回もこのケースでした。登録が失敗したと判断して、同じ園児を追加し直さないでください。

完了の目安：ホームの承認待ちが解消し、園児詳細・家族プロフィール・保護者アカウントに情報があり、保護者がログインできました。

15 家族の追加とコードの使い分け

操作する場所：職員の「保護者アカウント」と保護者の画面。

家族プロフィールと保護者アカウント

家族プロフィールは家族共通の台帳、保護者アカウントは本人がログインするための登録です。対象の保護者を明示的に紐付けると、氏名・連絡先・勤務先などの共通項目が連動します。

既存アカウントでは編集画面の「家族プロフィールの保護者と紐付ける」を確認します。保護者が利用開始後に「登録情報」から変更を申請した場合は、園が「プロフィール変更申請」で承認して反映します。

祖父や祖母もログインできるようにする

保護者アカウントの「新規登録」で、その人自身の名前・メール・本人確認に使う情報を登録します。所属家族を選び、閲覧を認める園児だけにチェックを付け、保存します。その後「認証管理」で招待または本人確認後の初回設定コードを送ります。

家族プロフィールの入力枠は現時点で保護者1・2です。祖父母を追加のログイン利用者として登録することと、プロフィールに保護者3・4の枠を増やすことは別です。同じ家族を選んだだけでは、全きょうだいへの閲覧権限は付きません。お迎え連絡先だけが必要なのか、アプリの閲覧も必要なのかを確認します。

メールとコードの期限

案内	期限と使い方
初回入力の招待リンクと登録コード	24時間。1回限り
初回入力を始めた後の入力画面	2時間
園の承認後のパスワード設定リンク	24時間。開いた後の設定画面は15分
保護者の初回設定・再設定コード	発行から24時間。6文字のコードでパスワードを設定
職員の有効化・再設定コード	30分

保護者の「初回設定コードを発行してメール送信」「再設定コードを発行してメール送信」では、コード入力URL・コード・ログインURLが届きます。再発行は60秒以上あけ、1日10回までです。再発行前のコードは無効になります。変更前に発行したコードは、メールに書かれた元の期限が有効です。

16 プッシュ通知の送信設定を用意する

操作する場所：SSH端末。アプリと保護者ログインの確認後に行います。

ここは新規導入した基準版向けの手動設定です。今回の稼働済み実機については第21章の記録を参照します。通知画面に「園側の通知設定は準備中」やVAPID未設定と出る場合、鍵と送信設定が必要です。Gmailの設定だけでは端末通知は有効になりません。

通知の秘密鍵をTrueNAS内に作る

第4～5章のBASE・IMAGE・APP_UID・APP_GIDを同じ端末で使します。秘密鍵ファイルが既にあれば、この生成操作を繰り返しません。以下は新規ファイル作成のみを許可しています。

```
sudo docker run --rm -i --network none --user 0:0 -v "$BASE/secrets:/keys" --entrypoint python "$IMAGE" - "$APP_UID" "$APP_GID" <<'PY'
import base64, os, sys
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ec
key = ec.generate_private_key(ec.SECP256R1())
pem = key.private_bytes(serialization.Encoding.PEM,
    serialization.PrivateFormat.PKCS8, serialization.NoEncryption())
fd = os.open('/keys/parent-push-vapid.pem',
    os.O_WRONLY | os.O_CREAT | os.O_EXCL | os.O_NOFOLLOW, 0o400)
with os.fdopen(fd, 'wb') as out:
    os.fchown(out.fileno(), int(sys.argv[1]), int(sys.argv[2]))
    out.write(pem)
public = key.public_key().public_bytes(serialization.Encoding.X962,
    serialization.PublicFormat.UncompressedPoint)
print('Public key:', base64.urlsafe_b64encode(public).decode().rstrip('='))
PY
```

表示されるのは公開鍵だけです。.env に HOIKUICT_PUSH_VAPID_PUBLIC_KEY='表示された公開鍵' と HOIKUICT_PUSH_VAPID_SUBJECT='mailto:施設の連絡先メール' を追記します。

次の章で設定を適用します。秘密鍵は再起動しても維持し、秘密情報のバックアップにも含めます。失うと、登録済み端末の再登録が必要になることがあります。

17 通知設定を適用して自分の端末へ送る

操作する場所：最初はSSH端末、その後は保護者端末。

Composeへ通知設定を反映する

新規導入で、まだ端末登録も未完了の通知もない状態で進めます。すでに業務通知が作られている環境では、未送信キューを管理担当者が確認してから有効化します。

compose.webpush.yaml の内容を、現在の compose.yaml のappサービスへ追加します。編集前に両ファイルを権限制限した場所へ保存します。通知用設定の詳細は [通知設定の技術手順](#) と [通知用Compose](#) にあります。

cd "\$STACK" の後に sudo vi compose.yaml で編集します。下の項目は app の environment 内に入れ、すでにある HOIKUICT_PUSH_TRANSPORT の行は置き換えます。字下げは周囲とそろえ、全角スペースやタブは使いません。

appに追加または変更する環境変数	値
HOIKUICT_PUSH_TRANSPORT	webpush
HOIKUICT_PUBLIC_ORIGIN	https://\${PILOT_HOSTNAME}
HOIKUICT_PUSH_VAPID_PUBLIC_KEY	\${HOIKUICT_PUSH_VAPID_PUBLIC_KEY}
HOIKUICT_PUSH_VAPID_PRIVATE_KEY	/run/secrets/parent-push-vapid.pem
HOIKUICT_PUSH_VAPID_SUBJECT	\${HOIKUICT_PUSH_VAPID_SUBJECT}

appのvolumesへ次の1行も追加します。既存のDB・添付・禁止パスワード一覧の行は残します。

```
- ${APP_SECRETS_PATH}/parent-push-vapid.pem:/run/secrets/parent-push-vapid.pem:ro
```

sha256sum compose.yaml の新しい値を .env の BACKUP_COMPOSE_SHA256 に入れ、検査してアプリを再作成します。

```
sudo docker compose --profile '*' config --quiet
sudo docker compose run --rm --no-deps --entrypoint python app -c 'from security_config import validate_runtime_security; validate_runtime_security(); print("Push settings OK")'
sudo docker compose up -d --no-deps app
sudo docker compose ps
```

端末の受信確認

保護者ログイン後「通知設定」を開き、「この端末で通知を受け取る」→通知を許可→「この端末へテスト通知を送る」の順に操作します。「プッシュ通知を利用する」もオンにして保存します。テストは操作中の1台に届きます。

iPhone・iPadはiOS/iPadOS 16.4以降でサイトをホーム画面へ追加し、そのアイコンから開いて登録します。

[WebKit公式説明](#)

画面の受付表示に加え、OS上の通知表示とタップ後の移動を確認します。現在の業務通知は「出欠確認のお願い」です。すべてのお知らせや日次連絡が自動でプッシュされるわけではありません。

18 バックアップと更新の基本

操作する場所：SSH端末とTrueNASの管理画面。停止時間を利用者へ案内して行います。

日々のデータを守る

DBは runtime/data、添付は runtime/storage にあります。両方が同じ時点でそろそろよう、アプリを停止して runtime全体のスナップショットを取ります。今回の構成での最小手順は次のとおりです。

```
cd "$STACK"
sudo docker compose stop cloudflared app
sudo zfs snapshot main/open-hoikuict-pilot/runtime@manual-20260906-1200
sudo docker compose start app
sudo docker compose ps
```

データセット名と末尾の日時は自分の環境に変更します。同名のスナップショットは再作成できません。appが healthyになったことを確認してからcloudflaredを開始します。追加のworkerを稼働させている場合は、それらの書き込みも停止します。

```
sudo docker compose start cloudflared
```

同じプール内のスナップショットだけでは、機器故障に備えられません。可搬バックアップの作成、別機器への複製、隔離環境への復元を確認します。[停止バックアップと復元の詳しい手順](#)

scripts.backup_runtime のCLIも利用できます。--quiesced は、アプリを停止済みかスナップショットの複製を読み取っていることを確認する指定です。コマンド自体がアプリを停止するわけではありません。

更新するときの順序

1. 新しいコミットのテスト結果と、DB変更の有無を確認します。
2. 現在の版、イメージID、Compose、.env、秘密鍵の保管先を記録します。
3. 停止中のDBと添付を保存し、必要な外部バックアップを確認します。
4. 新しいイメージを別にビルドし、設定を検査して切り替えます。
5. 健康状態、管理者ログイン、主要件数、保護者の閲覧を確認します。

GitHubのmainを更新しても、TrueNASで動いているアプリは自動では更新されません。また、DB変更後に古いイメージだけへ戻すと不整合になる場合があります。戻すときは対応するコード・DB・添付を一組で扱います。

19 接続とメールで困ったとき

SSHやsudoで止まる

Permission denied は接続先・ユーザー・鍵・パスワードを確認します。sudoで文字が見えないのは正常です。複数回失敗した場合は入力を繰り返す前に、接続先を確認します。

アプリがhealthyにならない

次を同じスタックの中で実行し、最初のエラーを確認します。

```
cd "$STACK"  
sudo docker compose ps  
sudo docker compose logs --tail=50 app
```

よくある原因は .env の仮値、秘密鍵や禁止パスワード一覧の読み取り権限、DBフォルダーの書き込み権限です。ログを共有する場合は、名前・メール・URL内のコードなどを除きます。docker inspect や .env の全文を貼らないでください。

Cloudflareで入れない

施設ホスト名が .env と一致するか、対象メールがAllowに入っているか、Team nameとAUDがそのAccessアプリの値かを確認します。AUDの赤いエラーは、入力後の確定操作も確認します。

Tunnelが接続済みでも、アプリまで到達できるとは限りません。502の場合はアプリの状態と、サービスURL <http://app:8000> を確認します。新しいシークレットウィンドウでAccessの許可・拒否を再確認します。

招待メールが届かない

1. 対象者の「認証管理」で宛先とメール送信履歴を確認します。
2. 迷惑メール、宛先の打ち間違い、受信側の制限を確認します。
3. SMTP接続先がMailpitのままになっていないか確認します。
4. Gmailの送信元・ユーザー名が一致し、アプリパスワードを使っているか確認します。

Googleアカウントのパスワード変更後は、アプリパスワードが失効します。必要なら再発行してアプリの設定を更新します。送信量が増える段階では、メールの上限と配送状況も確認してください。

20 登録と通知で困ったとき

提出できたのに園の台帳へ反映されない

「初回入力を受け付けました」は提出完了です。園の承認後に台帳へ反映します。管理者ホームを更新し、対象者の認証管理にある登録申請の履歴も確認します。

履歴にもない場合は、園と保護者が同じ施設URLを使っているか、対象のメールアドレスとアカウントが一致するかを確認します。再招待すると古い未承認申請が取り消されるため、確認前に何度も再送しないでください。

家族情報が二重になっている

家族プロフィールと保護者アカウントの明示的な紐付けを確認します。別々に作られたアカウントを名前の一致だけで自動統合しません。対象園児のチェックも確認します。

コードが使えない

最新のメールか、有効期限内か、既に使用していないかを確認します。Cloudflareの認証コード、登録用の長いコード、6文字の設定コードは別です。それぞれメールにあるURLで使います。

通知ボタンが押せない

園側の通知設定が準備中なら第16～17章へ進みます。端末側の通知権限が拒否されている場合は、ブラウザのサイト設定とOSの通知設定を見直します。iPhoneではホーム画面に追加したアイコンから開きます。

テスト通知の受付は表示されるが通知が見えない

OSの通知センター、集中モード、ブラウザの通知許可、通信状態を確認します。送信サービスの受付は端末表示の保証ではありません。Cloudflare Accessの認証が切れると、表示やクリックの報告だけが届かない場合もあります。

テスト通知は60秒に1回、1日10回までで、送信期限は5分です。明示的に保護者ログアウトすると、その端末の通知登録は無効になります。再ログイン後に端末を登録し直します。

相談時は「実行した章」「画面の文言」「発生時刻」「PCかスマートフォンか」「期待した動作」を伝えます。パスワードやリンク内の認証情報は伝えません。

21 今回の導入記録と最終確認

今回の実機では、次の順で準備と改善を進めました。

作業	2026年9月6日時点の確認
専用ブランチと空の保存先を準備	実施済み。既存デモとは別のpilotスタック
空DBでアプリを起動	実施済み。アプリとSQLiteの確認に成功
Cloudflare TunnelとAccess	設定し、職員ログイン画面への到達を確認
初期管理者	日本語入力エラーへの対応後、ログインした画面を確認
Gmail送信	hikarinomorihoikuen@gmail.com へ設定変更。招待受信と提出を確認
保護者の初回入力	子どもの名前とメールから招待し、提出を園側で確認
家族連動とホームの承認待ち	対応する変更を実機へ反映
保護者コードの24時間化と本番通知	コードと更新ファイルを準備。最新の実機反映・端末受信の完了報告は未確認
mainへの統合	1cc7dc8 まで反映。GitHub ActionsのLintとテストが成功
外部バックアップと隔離復元	本書作成時点では完了を確認できていない

今回の施設ホスト名は hikarinomori.hoikuict.net、runtimeは /mnt/main/open-hoikuict-pilot/runtime です。別施設へ導入するときは、その施設の値を使用します。

今回配置した通知更新ヘルパーは、既存実機の特定の旧版を検査してから更新するものです。新規導入の共通コマンドではありません。既存実機で未実行の場合に限り、当時案内した次のファイルが対象です。

```
sudo python3 /home/truenas_admin/open-hoikuict-push-24153f7/update-pilot.py
```

このヘルパーは旧版 1e47165 と当時の構成を確認します。構成不一致で停止したら、確認処理を外さず現在の版を調べます。新規に本書の基準版を導入した環境には実行しません。

導入完了のチェックシート

施設名：_____ 確認日：_____ 担当者：_____

確認	項目
☐	専用の保存先を使用し、初回起動前に空だった
☐	導入したコミット、イメージID、Composeハッシュを記録した
☐	アプリがhealthyで、再起動してもデータが保持される
☐	Cloudflareで許可した人が通り、許可外の人拒否される
☐	管理者が有効化、ログアウト、再ログインできる
☐	Gmailから招待メールが届き、送信元も正しい
☐	子どもの名前とメールだけで初回入力を依頼できる
☐	保護者が提出し、園のホームから承認できる
☐	承認後に園児・家族・保護者の情報が反映される
☐	保護者がパスワードを設定し、対象園児だけを閲覧できる
☐	クラスと入園予定日が正しく、当日の人数を説明できる
☐	追加の保護者にも必要な園児だけの権限を付けた
☐	使用する各端末で通知を登録し、表示とタップを確認した
☐	DBと添付を同じ時点でバックアップした
☐	別の機器・場所へ複製し、隔離環境への復元を確認した
☐	秘密情報の保管先と、停止・復旧時の担当者を決めた

未完了の項目と次の担当者：

困ったときに参照する資料

- ・ [TrueNASへの初期導入 技術者向け](#)
- ・ [家族連動と保護者招待の説明](#)
- ・ [通知の本番構成と受信確認](#)
- ・ [停止バックアップと隔離復元](#)
- ・ [本書の対象コミット](#)

外部サービスの画面名は変更されることがあります。本書の名称と画面が違う場合は、各章の公式リンクで該当する機能名を確認してください。